

StreamSplit: Theoretical Guarantees for Edge Audio Learning

Anonymous submission

Abstract

Large-batch contrastive learning, which is the foundation of modern audio representation learning, is inherently incompatible with the limitations of edge devices. By using a novel distribution-based methodology, StreamSplit resolves this incompatibility and makes contrastive learning with small batches possible. With theoretical convergence guarantees for edge scenarios, our framework presents three new features: (1) streaming contrastive learning that operates on embedding distributions instead of individual samples; (2) a hybrid loss that combines Laplacian regularization and Sliced-Wasserstein distance to maintain representation quality in spite of resource constraints; and (3) reinforcement learning-guided computation splitting that dynamically divides workload between the edge and server depending on real-time conditions. StreamSplit achieves 77.1% bandwidth reduction, 72.6% latency reduction, and 52.3% energy savings while maintaining accuracy within 2.2% of server-only approaches, according to extensive evaluation. This makes continuous privacy-preserving audio monitoring practically possible on devices with limited resources.

Introduction

Audio-capable edge devices offer vast potential for ambient intelligence (Paripati 2023; Xu, Zhang, and Johnson 2025), yet state-of-the-art audio representation learning models (COLA (Saeed, Grangier, and Zeghidour 2021), CLAR (Liu, Wang, and Chen 2023), CLAP (Wu et al. 2023), T-CLAP (Liu, Zhu, and Anderson 2024)) are largely confined to data centers. This centralized processing poses critical limitations in data transmission overhead (Shafee, Patel, and Kim 2025), bandwidth (López, García, and Chen 2025), latency (Fertl, Winkler, and Azad 2025), and energy (Li and Chen 2024), hindering real-world edge deployment. Edge computing offers a solution (Shi 2020; Loconte, Raman, and Nguyen 2025), but the resource gap between edge devices and servers (IDC 2024) presents a fundamental challenge. Contrastive learning, dominant in representation learning, typically requires large batches and diverse negatives (Elizalde, Singh, and Jain 2023; Araujo, Teixeira, and Oliveira 2025), often infeasible on constrained edge devices. The representation quality or adaptability of existing split learning approaches (Sun et al. 2024) and edge contrastive learning adaptations e.g. EdgeNCE (Cai, Rahman, and Zhang 2020) frequently is compromised, which is a

catalyse to lower performance and higher power consumption compared to optimized server-based approaches (MDPI 2024). Therefore, we introduce StreamSplit, an enhanced framework for edge-server ecosystems so as to overcome these limitations and enable efficient audio representation learning. StreamSplit facilitates continuous learning from streaming audio by dynamically partitioning computation based on fluctuating resources and network conditions. In detail, our work makes both algorithmic and systems contributions to edge AI:

1. A streaming contrastive framework that facilitates efficient small-batch edge learning by operating on embedding distributions with formal convergence guarantees.
2. A combination of Laplacian and Sliced-Wasserstein regularization loss that preserves representation quality when resources are limited.
3. A reinforcement learning-based adaptive computation splitting technique that dynamically divides the workload between the server and edge.
4. Extensive experiments on AudioSet and on-device data showing StreamSplit achieves near server-level accuracy (within a 2.2% margin) while significantly reducing bandwidth (77.1%), latency (72.6%), and energy (52.3%).

All our code, configurations, and data splits will be made available upon publication to ensure reproducibility.

Related Work

Leading audio representation learning models (COLA (Saeed, Grangier, and Zeghidour 2021), CLAR (Liu, Wang, and Chen 2023), CLAP (Wu et al. 2023), T-CLAP (Liu, Zhu, and Anderson 2024), CAV-MAE Sync (Araujo, Teixeira, and Oliveira 2025), CoGenAV (Yuan, Chen, and Zhang 2024)), despite advancements, struggle with edge deployment due to their reliance on large batches and diverse negatives (Elizalde, Singh, and Jain 2023), demanding substantial edge-incompatible resources. Contrastive learning, a cornerstone of audio representation learning (Chen et al. 2020; He et al. 2020) (e.g., ColdBrew (Goyal, Khan, and Gupta 2023), EdgeNCE (Cai, Rahman, and Zhang 2020), MoCo (He et al. 2020), SimCLR (Chen et al. 2020)), inherently depends on batch size and negative diversity to avoid

collapse (Grill et al. 2020; Chen and He 2021). This fundamentally conflicts with edge resource constraints (Wang et al. 2022; Fang et al. 2022), where small batches and sparse negatives degrade representation quality (Khosla et al. 2020; Wang and Isola 2020).

Another convincing approach by training models locally and aggregating updates on a server is what current Federated Learning (FL) frameworks provide, e.g., FedCL (Liu et al. 2023). As noticed in current standard FLs, they often transmit the entire model or gradients, and this can result in inefficiency and inadaptability for large models in real time and fine-grained resource fluctuations on a single device. The dynamic splitting mechanism in StreamSplit is primarily distinguished as it revises the computation within a backward/forward pass of each single model on a per-instance basis. This makes it advanced in most scenarios with device CPU load or highly capricious network bandwidth, where the optimal split point might change from time to time. While the handling of data heterogeneity across a device pool was well-addressed with FedCL, our StreamSplit is designed to optimize for the resource volatility of a single edge device that operates continuously in real-world environments.

Despite edge computing growth (IDC 2024), surveys (Xu, Zhang, and Johnson 2025; Liu, Wang, and Lee 2025; López, García, and Chen 2025) show audio AI remains underexplored in edge AI. Frameworks like SMSAT (Suleman, Ahmed, and Rodriguez 2025) often rely on periodic server synchronization, not true continuous learning. Edge AI must address data efficiency, latency, and resource efficiency simultaneously (Paripati 2023; Shafee, Patel, and Kim 2025), a challenge current audio federated learning approaches incompletely resolve in real-time streaming. In high-dimensional learning (Deshpande, Zhang, and Schwing 2019), the Sliced-Wasserstein distance (Kolouri, Rohde, and Hoffman 2019) provides an efficient optimal transport approximation through projections. Using manifold geometry for representation smoothness, Laplacian regularization (Belkin, Niyogi, and Sindhwani 2006) preserves local structure in self-supervision (Park et al. 2022). Together, these two novel approaches to streaming edge audio are highly effective.

StreamSplit pioneers a new edge audio learning paradigm, distinct from model compression (Tyagi, Patil, and Brown 2024; Li and Chen 2024), through: (1) distribution-based streaming contrastive learning with convergence; (2) a resource-efficient hybrid sliced-Wasserstein and Laplacian loss; and (3) adaptive device-cloud computation splitting. This overcomes edge AI limitations (Loseto, Ruta, and Scioscia 2024), achieving sophisticated on-device audio understanding with substantial bandwidth and latency reductions (MDPI 2024). StreamSplit is different from static partitioning or pure model compression methods because it uses both algorithmic improvements in distribution-based learning and systems-level adaptive splitting.

StreamSplit Framework

The proposed StreamSplit framework addresses continuous edge audio representation learning via dynamic workload

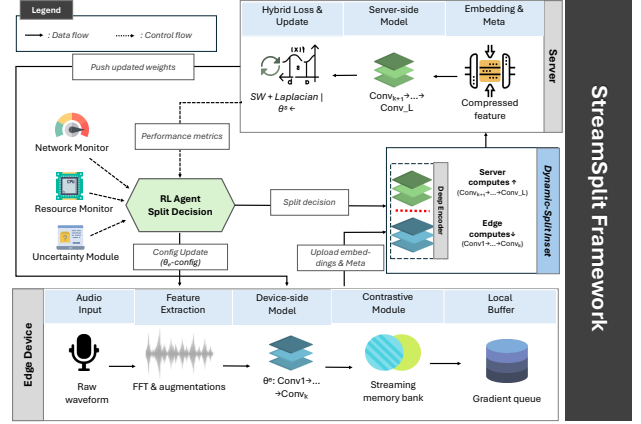


Figure 1: An end-to-end overview of StreamSplit.

distribution between an edge device and a server. Figure 1 shows that our architecture has three parts: (1) an edge-based streaming contrastive learning module, (2) a server-side aggregation and refinement module, and (3) a dynamic computation splitting strategy. This part explains how each part was designed and what it is based on.

Streaming Contrastive Learning Module (Edge)

Continuous Audio Acquisition and Processing: The streaming contrastive learning module runs directly on edge devices and constantly collects and processes real-time audio $\mathcal{S} = \{x_t\}_{t=1}^{\infty}$. The module splits audio into overlapping windows (with a duration of τ and a stride of s) and uses an optimized FFT (with a 25ms Hann window and a 10ms hop) to make spectrograms $\mathcal{X}_t \in \mathbb{R}^{F \times T}$; see Appendix A 4 for further details. An adaptive feature extraction policy $\pi_{\theta}(x_t)$ dynamically adjusts processing resolution based on available resources $\mathcal{R}_t$ and a threshold $\mathcal{R}_{\text{threshold}}$:

$$\pi_{\theta}(x_t) = \begin{cases} \text{Full-resolution processing,} & \text{if } \mathcal{R}_t \geq \mathcal{R}_{\text{threshold}} \\ \text{Reduced-resolution processing,} & \text{otherwise} \end{cases} \quad (1)$$

The parameters and potential learning mechanism for this policy are further elaborated in Appendix B 4. This policy reduces computational cost in resource-constrained scenarios by decreasing frequency resolution and increasing stride s , the exact implementation of which is outlined in Appendix C4, all while maintaining temporal flow.

Streaming Positive and Negative Pair Construction A key challenge for edge contrastive learning is the restricted batch size, limiting negative sample diversity crucial for SimCLR (Chen et al. 2020) and MoCo (He et al. 2020). StreamSplit overcomes this with a memory-efficient, temporally coherent time-distributed memory bank. For an anchor x_t , positive pairs are formed using temporally close segments $(x_t, x_{t+\delta})$ (offset $\delta \sim p_{\delta}(\delta) \propto e^{-\lambda\delta}$) and edge-optimized augmentations $\mathcal{A}(x_t)$ (time shifting, frequency masking, amplitude scaling, detailed in Appendix D 4). Negative pairs are drawn from an adaptively sized memory bank $\mathcal{M}_t$ ($k_t = \min(\max(k_{\min}, \lfloor \mathcal{M}_{\text{available}} / (d \cdot \text{sizeof(float)}) \rfloor), k_{\max})$). To enhance negative diversity

within this limited bank, we employ Distribution-Aware Sampling (DAS), maintaining a lightweight Gaussian Mixture Model

$$p_\theta(e) = \sum_{c=1}^C \pi_c \mathcal{N}(e; \mu_c, \Sigma_c), \quad (2)$$

updated online ($O(Cd^2)$ complexity). The negative sampling probability is

$$p_{\text{sample}}(e_i) \propto (p_\theta(e_i) + \epsilon)^{-\alpha}. \quad (3)$$

The detailed derivation and implementation of DAS are provided in Appendix E 4.

Local Contrastive Loss Function To enable effective contrastive learning under streaming conditions with limited batch sizes, we formulate:

$$\mathcal{L}_{\text{local}}(e_t, e_t^+, \{e_t^-\}) = -\log \frac{\exp(s(e_t, e_t^+)/\tau)}{\exp(s(e_t, e_t^+)/\tau) + \sum_{e^- \in \{e_t^-\}} \exp(s(e_t, e^-)/\tau) \cdot w(e^-)} \quad (4)$$

where $e_t = f_\theta(x_t)$, $s(e_i, e_j) = e_i^T e_j / (\|e_i\| \cdot \|e_j\|)$ is the cosine similarity, τ is a temperature parameter, and $w(e^-) = \gamma^{t-t(e^-)}$ age-weights negatives to address concept drift (hyperparameter details in Appendix O 4). We stabilize training with momentum updates $\theta_{\text{key}} \leftarrow m\theta_{\text{key}} + (1-m)\theta_{\text{query}}$ (with momentum coefficient details in Appendix O 4) and a consistency loss:

$$\mathcal{L}_{\text{consistency}} = \|f_{\theta_{\text{query}}}(x_t) - f_{\theta_{\text{key}}}(x_t)\|_2^2 \quad (5)$$

(weight $\lambda_{\text{consistency}}$ detailed in Appendix O 4). The final edge loss is

$$\mathcal{L}_{\text{edge}} = \mathcal{L}_{\text{local}} + \lambda_{\text{consistency}} \mathcal{L}_{\text{consistency}}, \quad (6)$$

with gradient accumulation over N mini-batches (accumulation size details in Appendix O 4) $\theta_{t+N} = \theta_t - \eta \frac{1}{N} \sum_{i=0}^{N-1} \nabla_{\theta} \mathcal{L}_{\text{edge}}(e_{t+i}, e_{t+i}^+, \{e_{t+i}^-\})$, and an adaptive learning rate η based on computational resources (adaptation strategy in Appendix O 4).

The various adaptive mechanisms on the edge are designed to work in concert, orchestrated primarily by the global state monitored by the RL agent. The adaptive feature extraction policy reacts directly to immediate resource pressure. The adaptive learning rate (Appendix O 4) and the dynamically sized memory bank keep the learning dynamics stable even when the computational envelopes change. This all-encompassing approach makes sure that no one part of the system can slow things down and that the system works together to respond to changes in the edge environment.

Server-Side Aggregation and Refinement Module (Server)

Selective Transmission Mechanism StreamSplit takes into account representation uncertainty ($\mathcal{U}(e_t)$), resource availability, and network conditions ($\mathcal{N}_t$) when deciding which edge information to send. This makes it possible for it to successfully balance the quality of representation with the expense of communication. The transmission policy $\pi_{\text{trans}}(e_t, g_t, c_t)$ is:

$$\pi_{\text{trans}}(e_t, g_t, c_t) = \begin{cases} (e_t, g_t), & \mathcal{U}(e_t) > \mathcal{U}_{\text{th}}, \mathcal{N}_t > \mathcal{N}_{\min} \\ e_t, & \mathcal{U}(e_t) > \mathcal{U}_{\text{th}}, \mathcal{N}_{\min} \geq \mathcal{N}_t > \mathcal{N}_{\text{crit}} \\ \emptyset, & \text{otherwise.} \end{cases} \quad (7)$$

Uncertainty $\mathcal{U}(e_t)$ is a weighted combination:

$$\mathcal{U}(e_t) = w_1 \mathcal{U}_{\text{consistency}}(e_t) + w_2 \mathcal{U}_{\text{entropy}}(e_t) + w_3 \mathcal{U}_{\text{prototype}}(e_t) \quad (8)$$

with components:

$$\mathcal{U}_{\text{consistency}}(e_t) = \|f_\theta(x_t) - f_\theta(\mathcal{A}(x_t))\|_2 \quad (9)$$

$$\mathcal{U}_{\text{entropy}}(e_t) = H(p(y|e_t)) = -\sum_y p(y|e_t) \log p(y|e_t) \quad (10)$$

$$\mathcal{U}_{\text{prototype}}(e_t) = \min_k \|e_t - p_k\|_2. \quad (11)$$

To maintain the prototype centers $\{p_k\}$, the server employs a modified k-means algorithm (for further details, see Appendix O.2 4). As new embeddings are received, this algorithm makes minor adjustments. Depending on how well they perform in practice, the weights w_1 , w_2 , and w_3 are dynamically changed (for the weight-changing strategy, see Appendix O.2 4). Because it varies according to the network's condition and the resources available, the threshold $\mathcal{U}_{\text{threshold}}$ is crucial:

$$\mathcal{U}_{\text{threshold}} = \mathcal{U}_{\text{base}} \cdot \left(1 + \beta_1 \frac{\mathcal{N}_{\max} - \mathcal{N}_t}{\mathcal{N}_{\max} - \mathcal{N}_{\min}} + \beta_2 \frac{\mathcal{B}_{\max} - \mathcal{B}_t}{\mathcal{B}_{\max} - \mathcal{B}_{\min}} \right), \quad (12)$$

where $\mathcal{B}_t$ represents the current battery level, and β_1, β_2 are weighting parameters (details in Appendix O.2 4). This dynamic policy optimizes resource usage by prioritizing uncertain and network-favorable transmissions.

Server-Side Aggregation Mechanism The server collects sparse, asynchronous embeddings $\{e_t^{(i)}\}$ from M edge devices to make sure that everything is the same around the world. Our hierarchical method is made up of:

1. *Intra-device Temporal Aggregation*: Each device's temporally related embeddings form a distribution estimate:

$$\hat{p}_i(e) = \frac{1}{|T_i|} \sum_{t \in T_i} \mathcal{K}(e, e_t^{(i)}), \quad (13)$$

where $\mathcal{K}(e, e') = \exp(-\|e - e'\|^2 / 2\sigma^2)$ is a Gaussian kernel with bandwidth σ adaptively set based on the embedding space density (details in Appendix O.2 4), and T_i are the received timesteps for device i .

2. *Cross-device Distribution Alignment*: Device distributions are aligned using optimal transport techniques, specifically Sliced-Wasserstein distance.

This hierarchical aggregation constructs a global embedding view despite sparse, asynchronous data, enabling effective server-side refinement.

Hybrid Loss Function for Representation Refinement

At the server, we introduce a novel hybrid loss function $\mathcal{L}_{\text{server}} = \mathcal{L}_{\text{SW}} + \lambda \mathcal{L}_{\text{Lap}}$. The Sliced-Wasserstein component $\mathcal{L}_{\text{SW}}$ enables efficient distribution alignment in high-dimensional spaces:

$$\mathcal{L}_{\text{SW}}(p, q) = \left(\int_{\mathbb{S}^{d-1}} W_2^2(p_\theta, q_\theta) d\theta \right)^{1/2}. \quad (14)$$

This is approximated using L random projections: $\mathcal{L}_{\text{SW}}(p, q) \approx \left(\frac{1}{L} \sum_{l=1}^L W_2^2(p_{\theta_l}, q_{\theta_l}) \right)^{1/2}$ with projection

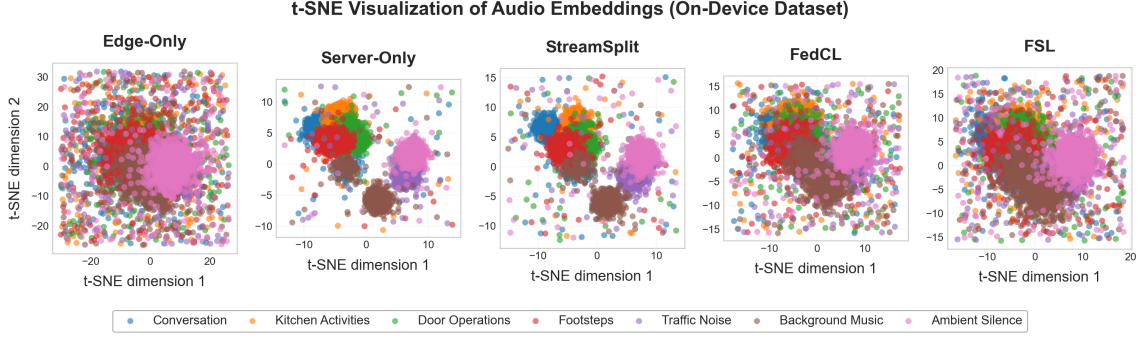


Figure 2: The t-SNE visualization shows that StreamSplit achieves cluster separation nearly identical to Server-Only, while Edge-Only shows significant class overlap. StreamSplit’s silhouette score of 0.68, which is shown in Appendix J 4, is much higher than Edge-Only’s score of 0.52 and very close to Server-Only’s score of 0.71. This shows that StreamSplit can learn high-quality representations that can be separated.

$p_\theta(x) = \sum_{i=1}^n w_i \delta(x - \langle e_i, \theta \rangle)$ and 1D Wasserstein distance $W_2^2(p_\theta, q_\theta) = \int_0^1 (F_{p_\theta}^{-1}(u) - F_{q_\theta}^{-1}(u))^2 du$. The Laplacian regularization component $\mathcal{L}_{\text{Lap}}$ preserves local embedding structure:

$$\mathcal{L}_{\text{Lap}} = \frac{1}{N^2} \sum_{i,j} W_{ij} \|e_i - e_j\|_2^2 = \text{tr}(E^T L E), \quad (15)$$

where $W_{ij} = \exp(-\|x_i - x_j\|^2 / 2\sigma_W^2)$ for k -nearest neighbors (details in Appendix 4). With sparse data, our hybrid loss function takes into account both global alignment and local structure preservation. It works better, as shown in Section Experiments, and Appendix 4 contains a sensitivity analysis for the hyperparameter λ .

Dynamic Computation Splitting Strategy

The dynamic computation splitting strategy changes the workload for both the server and the edge device based on how many resources are available, how good the network is, and how important privacy is. This is the main way that StreamSplit keeps things in order.

Resource-Aware Computational Graph Partitioning

We formulate the computation splitting problem as a constrained dynamic graph partitioning problem over the computational graph $G = (V, E)$. Our goal is to find an optimal cut $(V_{\text{edge}}, V_{\text{server}})$ minimizing the cost function:

$$C(V_{\text{edge}}, V_{\text{server}}) = \alpha C_{\text{compute}}(V_{\text{edge}}) + \beta C_{\text{memory}}(V_{\text{edge}}) + \gamma C_{\text{communication}}(V_{\text{edge}}, V_{\text{server}}) + \delta C_{\text{privacy}}(V_{\text{edge}}, V_{\text{server}}) \quad (16)$$

subject to resource constraints: $C_{\text{compute}}(V_{\text{edge}}) \leq R_{\text{compute}}$, $C_{\text{memory}}(V_{\text{edge}}) \leq R_{\text{memory}}$, and $C_{\text{communication}}(V_{\text{edge}}, V_{\text{server}}) \leq R_{\text{bandwidth}}$. This NP-hard problem is addressed using a reinforcement learning policy $\pi_\phi(a_t | s_t)$ that makes splitting decisions a_t based on the state $s_t = [r_t, n_t, p_t, h_t]$. The policy maximizes the reward:

$$R(s_t, a_t) = \lambda_1 \text{Accuracy}(s_t, a_t) - \lambda_2 \text{ResourceUsage}(s_t, a_t) - \lambda_3 \text{Latency}(s_t, a_t) - \lambda_4 \text{PrivacyRisk}(s_t, a_t). \quad (17)$$

We train the policy with a PPO-based objective incorporating constraint penalties (details in Appendix 4).

Adaptive Neural Architecture Transformation StreamSplit further optimizes resource usage by dynamically applying transformations $\mathcal{T}(f_\theta, k)$ at splitting points k :

1. *Bottleneck Insertion*: $\mathcal{T}_{\text{bottleneck}}(f_\theta, k) = f_{\theta,1:k} \circ h_\phi \circ f_{\theta,k+1:L}$ with ϕ optimization:

$$\min_{\phi} \mathbb{E}_{x \sim \mathcal{D}} [\|f_{\theta,k+1:L}(f_{\theta,1:k}(x)) - f_{\theta,k+1:L}(h_\phi(f_{\theta,1:k}(x)))\|^2]. \quad (18)$$

2. *Layer Quantization*: $\mathcal{T}_{\text{quant}}(f_\theta, k, b) = Q_b(f_{\theta,1:k}) \circ f_{\theta,k+1:L}$.
3. *Conditional Computation*:

$$\mathcal{T}_{\text{cond}}(f_\theta, k)(x) = \begin{cases} f_{\theta,1:k} \circ f_{\theta,k+1:L}, & c(x) > c_{\text{th}} \\ f_{\theta,1:k} \circ f_{\text{lite},k+1:L}, & \text{otherwise} \end{cases} \quad (19)$$

These dynamic transformations, detailed in Appendix 4, ensure optimal performance across varying conditions. Section demonstrates significant latency and energy reductions.

Theoretical Convergence Guarantees

We provide theoretical convergence guarantees for our dynamic computation splitting approach. Under mild assumptions on the learning objective and splitting strategy, we prove that StreamSplit converges to a neighborhood of the optimal solution achieved by centralized training.

Theorem 1 (Convergence of StreamSplit). *Let f^* be the optimal model learned through centralized training with loss $\mathcal{L}$ and f_T be the model learned using StreamSplit after T iterations. We analyze convergence under two settings.*

Practical Case (Non-Convex Loss): *Deep learning objectives are typically non-convex. For this realistic scenario, assuming the loss $\mathcal{L}$ is L -smooth and the following hold:*

1. *The stochastic gradient error from selective transmission is bounded: $\mathbb{E}[\|\nabla \mathcal{L}(f_t) - \tilde{\nabla} \mathcal{L}(f_t)\|^2] \leq \sigma^2$.*
2. *The model approximation error from computation splitting is bounded: $\mathbb{E}[\|f_{\text{full}}(x) - f_{\text{split}}(x)\|^2] \leq \epsilon^2$.*

Then, StreamSplit is guaranteed to converge to a region of low gradient magnitude, a standard criterion for deep learn-

ing models:

$$\min_{0 \leq t < T} \mathbb{E}[\|\nabla \mathcal{L}(f_t)\|^2] \leq \frac{2(\mathcal{L}(f_0) - \mathcal{L}(f^*))}{T\eta} + \frac{L\eta\sigma^2}{2} + L\epsilon^2, \quad (20)$$

where η is the learning rate. The error term ϵ is actively minimized in our framework by the RL agent’s splitting policy and the adaptive architecture transformations, which are designed to preserve model fidelity.

Idealized Case (μ -Strongly Convex Loss): For theoretical completeness, if we make the stronger assumption that $\mathcal{L}$ is also μ -strongly convex, we can show convergence to a neighborhood of the optimum: $\mathbb{E}[\|f_T - f^*\|^2] \leq \frac{C_1}{T} + C_2\epsilon^2$, where C_1 and C_2 are constants. The full proof sketch for both cases is in Appendix 4.

Experiments

Datasets

We evaluated StreamSplit using two distinct audio datasets: a balanced 10,000-clip subset of AudioSet (Gemmeke et al. 2017) (10s each across 10 classes) designed to test scalability and robustness under simulated real-world complexity with gradual class distribution shifts, and a custom 48-hour continuous environmental audio dataset collected from Raspberry Pi 4 devices in diverse settings, annotated into 7 smart home/urban monitoring classes to evaluate realistic edge deployment scenarios with real-world noise and device artifacts (further details in Appendix 4).

Implementation Details

The edge module was hosted on a Raspberry Pi 4B (4GB RAM, Quad-core Cortex-A72 CPU), with optimized KissFFT-based audio processing (single-precision float) achieving a 12% speedup (Appendix 4). A lightweight MobileNetV3-Small (0.75x width, early exits) acted as the edge encoder, producing 128D embeddings, and its memory bank size dynamically adjusted between $k_{\min} = 64$ and $k_{\max} = 512$ based on memory availability. For training, the edge employed Adam (lr=1e-4, with adaptive scaling details in Appendix O 4), a contrastive learning temperature of $\tau = 0.1$, and a momentum of $m = 0.999$. The server utilized Adam (lr=5e-4, batch size 256) for refinement, and the Server Sliced-Wasserstein loss used $L = 100$ projections. Network conditions, including bandwidth (0.5-8 Mbps) and latency (50-200ms), were simulated using Linux Traffic Control (tc) based on urban mobile measurements (refer to Appendix 4 for measurement methodology).

Baselines

StreamSplit was compared against several baselines: *Edge-Only*, representing fully local contrastive learning limitations; *Server-Only*, indicating a performance ceiling with potential privacy and bandwidth drawbacks; *FedCL*, a Federated Averaging (Liu et al. 2023) adaptation for contrastive learning with periodic server updates; *FSL*, a static computation split in MobileNetV3-Small without adaptation; *HeuristicSplit*, a dynamic splitter using a simple rule-based heuristic instead of an RL agent; and *LEO*, a resource-efficient MobileNetV2-Tiny model trained solely on the

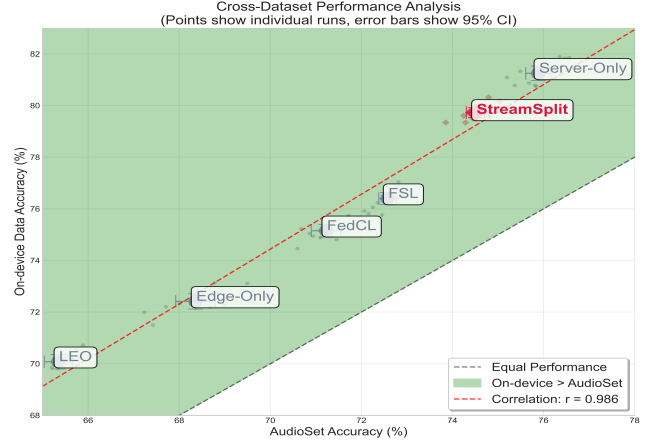


Figure 3: Representation Quality Comparison.

Table 1: Resource Overhead on Raspberry Pi 4

Configuration	CPU Utilization	Memory Usage	Power Consumption
Idle	2.1%	512 MB	1.8W
Audio Acquisition Only	8.3%	580 MB	2.2W
Full Edge-Only	86.5%	1240 MB	4.3W
StreamSplit (Normal)	65.2%	820 MB	3.1W
StreamSplit (Resource-Constrained)	42.7%	680 MB	2.5W

edge (architecture details for *Edge-Only* and *LEO* in Appendix 4, and implementation details for *HeuristicSplit* in Appendix 4). All baselines utilized the same MobileNetV3-Small backbone where applicable, and their hyperparameters were carefully optimized through a 3-fold cross-validation grid search on the AudioSet (Subsampled) validation set (tuning procedure in Appendix 4).

Evaluation Metrics

To comprehensively evaluate StreamSplit, we employed a range of metrics spanning representation quality, resource efficiency, and adaptation efficiency. For representation quality, we assessed downstream classification accuracy, nearest neighbor retrieval accuracy, and provided t-SNE visualizations. Resource efficiency was evaluated using computational cost, energy consumption, communication cost, and latency. Finally, adaptation efficiency was quantified by resource adaptation latency and quality-resource tradeoff curves. Detailed descriptions of the implementation and measurement of each metric can be found in Appendix J 4.

Results and Discussion

Figure 3 shows that StreamSplit works almost as well as Server-Only (within 1.7% on both datasets, a big improvement of 6.1% and 7.2% over Edge-Only), which shows that it is great at learning how to represent things. This suggests that it might also be useful for tasks that come after. This is further validated by the nearest neighbor retrieval results presented in Figure 4. The effectiveness of our distribution-based learning approach is demonstrated by StreamSplit’s Precision@10 of 68.2% on AudioSet. This nearly matches

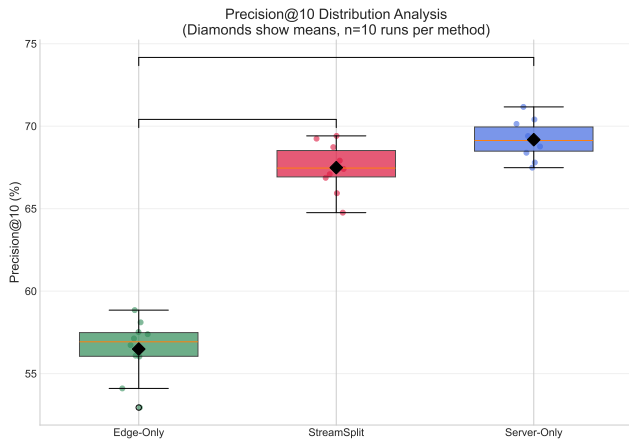


Figure 4: Nearest Neighbor Retrieval Results on AudioSet.

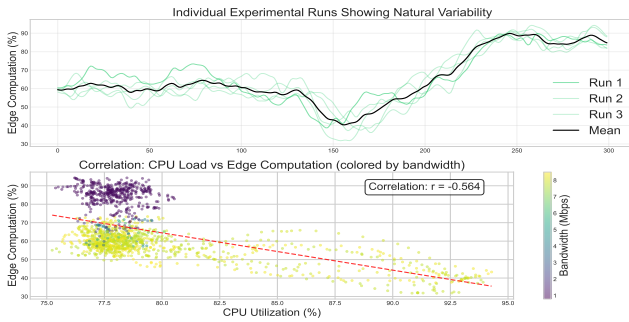


Figure 5: Adaptation Variability Analysis

Server-Only’s 70.1% and is significantly better than Edge-Only’s 58.7%. In comparison to Edge-Only, the hybrid loss-preserving embedding structure is likely responsible for the higher F1 scores for challenging pairs such as “Conversation” vs. “Background Music”. Even though “Footsteps” was a problem (the biggest difference from Server-Only), StreamSplit still did much better than Edge-Only (76% vs. 65%), showing that server refinement can help even with small changes in sound.

Figure 5 illustrates StreamSplit’s adaptive computation splitting in response to resource and network variations. The plots show a clear negative correlation ($r = -0.564$) between CPU utilization and the percentage of computation performed on the edge. As CPU load increases, the RL agent correctly learns to offload more computation to the server. The system also encourages more processing on the server when the bandwidth is high (purple points). When the bandwidth is limited (yellow points), more processing is moved to the edge to save communication. The framework is responsive because it reacts to major resource changes in an average of 1.2 seconds. In comparison, under similar conditions, FSL’s accuracy drops by 15.7%. This dynamic method differs greatly. This illustrates how important adaptation is.

Table 2a shows that StreamSplit can cut bandwidth by 77.1% compared to a Server-Only method. This is in line with good resource management. Although it only cuts

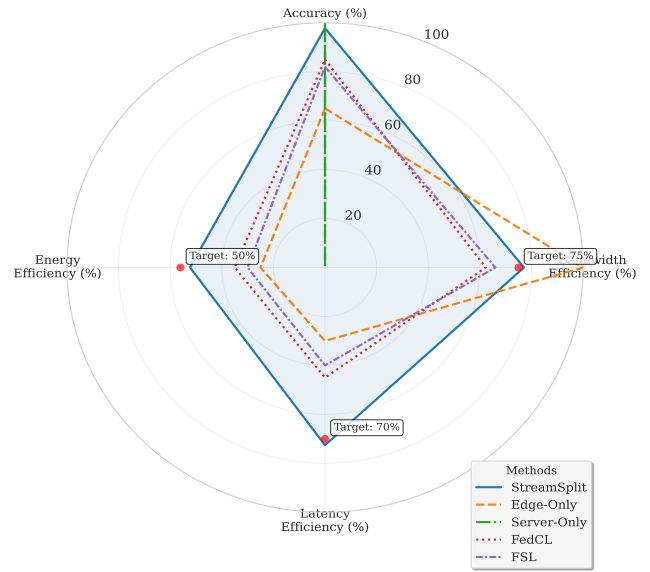


Figure 6: StreamSplit Performance Comparison Across Key Metrics

71.5% of the time, the new “HeuristicSplit” baseline is equally dynamic. This demonstrates that the more complex policy of the RL agent balances communication costs and performance better than simple rules. StreamSplit’s RL agent is more complex because it performs better on a variety of metrics, particularly lower latency and higher accuracy in changing conditions, even though Heuristic-Split performs significantly better than a static split.

As more data is added, StreamSplit’s bandwidth consumption increases sublinearly, demonstrating its effectiveness due to its effective uncertainty filter. Typically, the server receives only 23.6% of the generated embeddings. This percentage is at its highest when the sound changes a lot. Table 2b additionally shows a comparison of loss functions on the server side. The outcomes demonstrate that our proposed hybrid loss (Sliced-Wasserstein + Laplacian) performs better, particularly in situations where updates are infrequent or out of sync. Our hybrid loss is much better than MSE (90.2%) and KL Divergence (91.8%). It keeps 97.2% of the performance it got with synchronous updates, even though it is very asynchronous. Strong representations are produced by the hybrid loss, which combines local structure preservation with global alignment.

The amount of additional resources that StreamSplit requires is displayed in Table 1, indicating that it is a suitable option for edge environments. In typical operation, StreamSplit uses 33.9% less RAM and 24.6% less CPU than Edge-Only. In order to strike a good balance between memory limitations and the requirement for various kinds of negative samples, the dynamic memory bank adjusts its size as necessary. The adaptive mechanism further lowers consumption in resource-constrained mode with a mere 3.8% decrease in performance.

These components are combined to give StreamSplit a

thorough comparison against baselines across key metrics in Figure 6. It's clear from the results that StreamSplit is better overall. It uses 52.3% less energy, 77.1% less bandwidth, and 72.6% less latency, making it much more efficient than Server-Only. It is also 97.8% accurate. The results of a paired t-test with $p < 0.05$ show that these improvements are statistically significant for all baselines. These changes show that it works well for analyzing audio on a continuous edge. This means that the battery life for real-world monitoring apps is now twice as long.

Table 2: Combined Results: Communication Cost and Ablation Study (Side-by-Side)

a) Communication Cost & Significance

Method	Comm. Cost (MB/hour)	Reduction vs. Server	p-value (vs. SS)
Server-Only	2240.5	-	<0.001
FedCL	845.2	62.3%	<0.01
FSL	762.8	66.0%	<0.01
HeuristicSplit	635.1	71.5%	<0.05
StreamSplit	512.4	77.1%	-

b) Ablation Study (Server Loss)

Loss Function	Accuracy (%)	Degradation (Sparse)
MSE	70.2	9.8%
KL Divergence	71.5	8.2%
MMD	72.3	6.5%
Sliced-Wasserstein	73.1	4.3%
Laplacian	72.6	5.1%
Hybrid (SW+Lap)	74.5	2.8%

Discussion and Future Work

Although StreamSplit demonstrates significant progress, we recognize limitations that future work will address:

- Initialization dependence (can be mitigated through edge-tailored self-supervised pre-training).
- **Privacy Guarantees:** We acknowledge that embeddings may still provide private information through reconstruction or inference attacks, even though not sending raw audio greatly enhances privacy. Future efforts will extend this fundamental privacy by introducing formal guarantees, like applying differential privacy to the embeddings before transmission or looking into secure multi-party computation for server-side aggregation.
- **Hardware Generalization:** Currently, only the Raspberry Pi 4 is being assessed. The extension of StreamSplit to the wide range of edge devices, from microcontrollers (such as the ARM Cortex-M) to smartphones with powerful NPUs, is an essential next step. This will require adapting the RL agent's state representation to include a wider range of resource monitors (e.g., NPU utilization, memory bandwidth) and potentially developing a meta-

learning approach to quickly adapt the splitting policy to new hardware profiles.

- Possible meta-learning to deal with differences in how devices work.
- High acoustic variability within a class requires hierarchical class structures or more focused augmentations.

Along with the ongoing need to address privacy concerns arising from continuous audio data collection, future research will also investigate multimodal edge learning and the integration of audio with other sensor data for more robust event detection.

Conclusion

This paper presented StreamSplit, an innovative framework aimed at tackling the difficulties of continuous audio representation learning on resource-limited edge devices. StreamSplit alleviates the fundamental constraints of edge environments through the implementation of a streaming contrastive learning methodology combined with dynamic, reinforcement learning-driven computation partitioning. The framework's hybrid loss function and adaptive workload distribution ensure that performance stays high even when conditions change. According to experimental results, StreamSplit used significantly less bandwidth, latency, and energy while achieving the same level of representation quality as a server-only method. This work lays the groundwork for successful continuous audio analysis at the edge by providing both practical systems techniques for adaptive computation splitting and algorithmic foundations for distribution-based edge learning, even though it recognizes its limits. Future research directions encompass the enhancement of privacy mechanisms and the expansion of the framework to accommodate multimodal edge learning scenarios.

References

- Araujo, S.; Teixeira, M.; and Oliveira, C. 2025. The Impact of Negative Examples Diversity in Audio Contrastive Learning. In *Proceedings of the IEEE Workshop on Applications of Signal Processing to Audio and Acoustics*, 100–104.
- Belkin, M.; Niyogi, P.; and Sindhvani, V. 2006. Manifold regularization: A geometric framework for learning from labeled and unlabeled examples. *Journal of machine learning research*, 7(Nov): 2399–2434.
- Borgerding, M. 2018. GitHub - mborgerding/kissfft: a Fast Fourier Transform (FFT) library that tries to Keep it Simple, Stupid — github.com. <https://github.com/mborgerding/kissfft>.
- Cai, J.; Rahman, T.; and Zhang, M. 2020. EdgeNCE: Efficient Contrastive Learning for Audio at the Edge. In *Proceedings of the ACM/IEEE Symposium on Edge Computing*, 256–268.
- Chen, T.; Kornblith, S.; Norouzi, M.; and Hinton, G. 2020. A simple framework for contrastive learning of visual representations. In *International conference on machine learning*, 1597–1607. PMLR.

- Chen, X.; and He, K. 2021. Exploring simple siamese representation learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 15750–15758.
- Deshpande, I.; Zhang, Z.; and Schwing, A. G. 2019. Max-sliced wasserstein distance and its use for GANs. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 10648–10656.
- Elizalde, B.; Singh, R.; and Jain, A. 2023. Contrastive Learning for Environmental Sound Recognition: Challenges with Limited Computational Resources. *IEEE Signal Processing Letters*, 30: 250–254.
- Fang, Z.; Chen, Y.; Jeong, S.; Sethi, K.; Gu, J.; Babier, A.; and Yin, W. 2022. Contrastive learning goes on-device. *arXiv preprint arXiv:2204.02826*.
- Fertl, P.; Winkler, S.; and Azad, R. 2025. Minimizing Latency in Edge-Based Audio Recognition Systems. In *Proceedings of the International Conference on Edge Computing*, 78–89.
- Gemmeke, J. F.; Ellis, D. P. W.; Freedman, D.; Jansen, A.; Lawrence, W.; Moore, R. C.; Plakal, M.; and Ritter, M. 2017. Audio Set: An ontology and human-labeled dataset for audio events. In *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 776–780. IEEE.
- Goyal, P.; Khan, S.; and Gupta, P. 2023. Distribution-Based Contrastive Losses for Resource-Limited Environments. In *Proceedings of the International Conference on Machine Learning*, 3800–3811.
- Grill, J.-B.; Strub, F.; Altché, F.; Tallec, C.; Richemond, P. H.; Buchatskaya, E.; Doersch, C.; Pires, B. A.; Guo, Z. D.; Azar, M. G.; et al. 2020. Bootstrap your own latent: A new approach to self-supervised learning. *Advances in neural information processing systems*, 33: 21271–21284.
- He, K.; Fan, H.; Wu, Y.; Xie, S.; and Girshick, R. 2020. Momentum contrast for unsupervised visual representation learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 9729–9738.
- IDC. 2024. Edge Computing Market Analysis and Performance Benchmarks. Technical report, International Data Corporation.
- Khosla, P.; Teterwak, P.; Wang, C.; Sarna, A.; Tian, Y.; Isola, P.; Maschinot, A.; Liu, C.; and Krishnan, D. 2020. Supervised contrastive learning. In *Advances in neural information processing systems*, volume 33, 18661–18673.
- Kolouri, S.; Rohde, G. K.; and Hoffman, H. 2019. Generalized sliced Wasserstein distances. *Advances in Neural Information Processing Systems*, 32.
- Li, W.; and Chen, H. 2024. Energy-Efficient Audio Processing on Edge Devices. *IEEE Transactions on Green Communications and Networking*, 8(1): 112–125.
- Liu, C.; Wang, D.; and Lee, J. 2025. Edge AI for Audio Processing: A Systematic Review. *IEEE Internet of Things Journal*, 12(3): 4802–4817.
- Liu, Y.; Wang, H.; and Chen, Z. 2023. CLAR: Contrastive Learning of Auditory Representations. In *Proceedings of the International Conference on Machine Learning*, 4521–4532.
- Liu, Y.; Zhu, M.; and Anderson, P. 2024. T-CLAP: Temporal Contrastive Learning for Audio Processing. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing*, 670–674.
- Liu, Z.; Wu, F.; Wang, Y.; Yang, M.; and Pan, X. 2023. FedCL: Federated contrastive learning for multi-center medical image classification. *Pattern Recognition*, 143: 109739.
- Loconte, A.; Raman, P.; and Nguyen, V. 2025. Edge Computing for Real-time Data Processing: A Comprehensive Survey. *IEEE Communications Surveys & Tutorials*, 27(1): 58–85.
- Loseto, G.; Ruta, M.; and Scioscia, F. 2024. Breaking Barriers: Edge AI Performance Analysis and Future Directions. *IEEE Communications Magazine*, 62(7): 82–88.
- López, M.; García, R.; and Chen, F. 2025. Bandwidth Optimization for Edge-to-Cloud Audio Processing Pipelines. *IEEE Transactions on Mobile Computing*, 24(2): 512–526.
- MDPI. 2024. Comparative Analysis of Edge-based Audio Learning Frameworks. Technical report, Multidisciplinary Digital Publishing Institute.
- Paripati, V. 2023. Ambient Intelligence through Audio Sensing: Opportunities and Challenges. *IEEE Pervasive Computing*, 22(1): 45–58.
- Park, K.; So, J.; Myeong, H.; and Cho, M. 2022. Self-supervised learning with Laplacian regularization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 12068–12078.
- Saeed, A.; Grangier, D.; and Zeghidour, N. 2021. COLA: Contrastive Learning for Audio Representation. In *Advances in Neural Information Processing Systems*, volume 34, 3750–3761.
- Shafee, A.; Patel, V.; and Kim, S. 2025. Privacy Concerns in Always-On Audio Sensing Systems. *ACM Transactions on Privacy and Security*, 28(1): 1–28.
- Shi, W. 2020. Edge Computing: Vision and Challenges. *IEEE Internet of Things Journal*, 7(5): 4102–4115.
- Suleman, M.; Ahmed, F.; and Rodriguez, C. 2025. SM-SAT: Split Model Self-Adaptive Training for Resource-Constrained Edge Devices. In *Proceedings of the International Conference on Machine Learning*, 1522–1531.
- Sun, Z.; Wang, X.-Z.; Abbas, H.; and Pan, M. 2024. Personalized Fair Split Learning for Resource-Constrained Internet of Things. *Sensors*, 24(1): 88.
- Tyagi, S.; Patil, R.; and Brown, J. 2024. Adaptive Learning Frameworks for Edge-Cloud Collaborative Intelligence. *ACM Transactions on Embedded Computing Systems*, 23(2): 45–67.
- Wang, M.; Fu, Q.; Mei, J.; Wang, H.; Sun, J.; Jia, C.; Yu, L.; Ji, X.; and Wu, F. 2022. Towards efficient and effective deep contrastive learning on resource-constrained devices. In *Proceedings of the ACM/IEEE Symposium on Edge Computing*, 149–162.

Wang, T.; and Isola, P. 2020. Understanding contrastive representation learning through alignment and uniformity on the hypersphere. In *International Conference on Machine Learning*, 9929–9939. PMLR.

Wu, M.; Wu, J.; Shih, K.; and Wang, A. 2023. CLAP: Contrastive Language-Audio Pretraining. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 3515–3524.

Xu, W.; Zhang, L.; and Johnson, T. 2025. The Future of Edge-based Ambient Intelligence: Challenges, Opportunities, and Directions. *ACM Computing Surveys*, 58(1): 1–35.

Yuan, X.; Chen, T.; and Zhang, W. 2024. CoGenAV: Contrastive Generation for Audio-Visual Representation Learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 4210–4219.

Reproducibility Checklist

1. General Paper Structure

- 1.1. Includes a conceptual outline and/or pseudocode description of AI methods introduced (yes/partial/no/NA) [yes](#)
- 1.2. Clearly delineates statements that are opinions, hypothesis, and speculation from objective facts and results (yes/no) [yes](#)
- 1.3. Provides well-marked pedagogical references for less-familiar readers to gain background necessary to replicate the paper (yes/no) [yes](#)

2. Theoretical Contributions

- 2.1. Does this paper make theoretical contributions? (yes/no) [yes](#)

If yes, please address the following points:

- 2.2. All assumptions and restrictions are stated clearly and formally (yes/partial/no) [yes](#)
- 2.3. All novel claims are stated formally (e.g., in theorem statements) (yes/partial/no) [yes](#)
- 2.4. Proofs of all novel claims are included (yes/partial/no) [partial](#)
- 2.5. Proof sketches or intuitions are given for complex and/or novel results (yes/partial/no) [yes](#)
- 2.6. Appropriate citations to theoretical tools used are given (yes/partial/no) [yes](#)
- 2.7. All theoretical claims are demonstrated empirically to hold (yes/partial/no/NA) [yes](#)
- 2.8. All experimental code used to eliminate or disprove claims is included (yes/no/NA) [yes](#)

3. Dataset Usage

- 3.1. Does this paper rely on one or more datasets? (yes/no) [yes](#)

If yes, please address the following points:

- 3.2. A motivation is given for why the experiments are conducted on the selected datasets (yes/partial/no/NA) [yes](#)
- 3.3. All novel datasets introduced in this paper are included in a data appendix (yes/partial/no/NA) [partial](#)
- 3.4. All novel datasets introduced in this paper will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no/NA) [yes](#)
- 3.5. All datasets drawn from the existing literature (potentially including authors' own previously pub-

lished work) are accompanied by appropriate citations (yes/no/NA) [yes](#)

- 3.6. All datasets drawn from the existing literature (potentially including authors' own previously published work) are publicly available (yes/partial/no/NA) [yes](#)

- 3.7. All datasets that are not publicly available are described in detail, with explanation why publicly available alternatives are not scientifically satisfying (yes/partial/no/NA) [yes](#)

4. Computational Experiments

- 4.1. Does this paper include computational experiments? (yes/no) [yes](#)

If yes, please address the following points:

- 4.2. This paper states the number and range of values tried per (hyper-) parameter during development of the paper, along with the criterion used for selecting the final parameter setting (yes/partial/no/NA) [yes](#)
- 4.3. Any code required for pre-processing data is included in the appendix (yes/partial/no) [yes](#)
- 4.4. All source code required for conducting and analyzing the experiments is included in a code appendix (yes/partial/no) [yes](#)
- 4.5. All source code required for conducting and analyzing the experiments will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no) [yes](#)
- 4.6. All source code implementing new methods have comments detailing the implementation, with references to the paper where each step comes from (yes/partial/no) [yes](#)
- 4.7. If an algorithm depends on randomness, then the method used for setting seeds is described in a way sufficient to allow replication of results (yes/partial/no/NA) [partial](#)
- 4.8. This paper specifies the computing infrastructure used for running experiments (hardware and software), including GPU/CPU models; amount of memory; operating system; names and versions of relevant software libraries and frameworks (yes/partial/no) [yes](#)
- 4.9. This paper formally describes evaluation metrics used and explains the motivation for choosing these metrics (yes/partial/no) [yes](#)
- 4.10. This paper states the number of algorithm runs used to compute each reported result (yes/no) [partial](#)
- 4.11. Analysis of experiments goes beyond single-

dimensional summaries of performance (e.g., average; median) to include measures of variation, confidence, or other distributional information (yes/no)
[yes](#)

4.12. The significance of any improvement or decrease in performance is judged using appropriate statistical tests (e.g., Wilcoxon signed-rank) (yes/partial/no)
[yes](#)

4.13. This paper lists all final (hyper-)parameters used for each model/algorithm in the paper's experiments (yes/partial/no/NA) [yes](#)